

Litter Detection Using YOLOv8 on the TACO Dataset

Syed Abdullah

Student at National University of Computer and Emerging Sciences (NUCES)

abdullahsyed2005@gmail.com

Abstract—Litter pollution is a growing environmental concern, necessitating automated detection systems for monitoring and cleanup. This project explores object detection using YOLOv8 [1] on the TACO dataset [2], aiming to identify and classify different types of litter in images. Initial experiments used the full TACO dataset (60 classes) with severe class imbalance. Later, a reduced and refined dataset with 18 classes was used to improve class balance. Multiple models were trained and evaluated; results show notable improvements in mAP, precision, and recall after dataset refinement and careful hyperparameter tuning. This report documents dataset preparation, model training methodology, experiments, and conclusions.

Index Terms—litter detection, object detection, YOLOv8, TACO dataset, computer vision, dataset balancing

I. INTRODUCTION

Litter pollution affects ecosystems, cityscapes, waste-management systems, and public health. Reliance on manual inspection makes monitoring slow, inconsistent, and resource intensive. Modern computer vision methods—particularly deep learning-based object detectors—offer scalable alternatives that can assist in automated waste detection and environmental maintenance.

YOLO (You Only Look Once) models are widely adopted for real-time object detection because they offer strong accuracy with low inference latency. The TACO (Trash Annotations in Context) dataset provides labeled images of many types of waste items and serves as a suitable benchmark for evaluating such models.

However, the original TACO dataset contains 60 highly imbalanced classes. Many classes contain very few samples, causing unstable gradients and poor per-class performance in early experiments. To address this, a refined version containing 18 classes with more uniform sample counts was created. This modification produced substantially better detection performance and more reliable training behavior.

II. PROPOSED METHODOLOGY

The methodology follows a structured approach, including dataset preparation, annotation conversion, model selection, and comparative experimentation.

A. Dataset Preparation

The original TACO dataset [3] is annotated in COCO format. YOLOv8 requires label files in YOLO format, so annotations were converted accordingly. Each image was paired

with a text file containing the class index and normalized bounding-box coordinates.

Preprocessing steps included:

- Removing corrupted or unreadable images.
- Standardizing directory structure for images and labels.
- Applying a 70/15/15 train-validation-test split.
- Using augmentations such as color jitter, flips, and rotations. Mosaic and mixup were applied selectively.

Because severe class imbalance affected learning, an alternative 18-class refined dataset was also used. This improved learning stability and boosted detection accuracy across most classes.

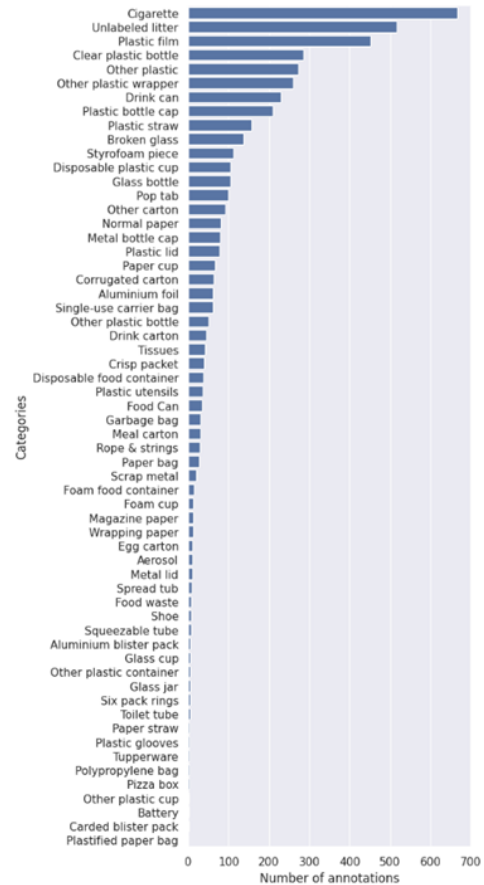


Fig. 1: Example class distribution from the TACO dataset [4]

B. Model Selection

YOLOv8 was selected due to its strong performance and efficient training pipeline. Multiple variants were trained. All models were trained on epochs=100, batch size=16, and image size= 640.

All experiments were logged through MLflow and tracked via DagsHub for versioning of models, metrics, and configuration.

C. Training Setup

Table I summarizes the principal training configurations used.

TABLE I: Model training configurations (summary)

Name	Base Model	Epochs	Batch	Img	LR	Optimizer	Augmentation
yolo8n_baseline	yolo8n.pt	100	16	640	0.01	SGD	None
yolo8s_baseline	yolo8s.pt	100	16	640	0.01	SGD	None
yolo8s_adam_optimizer	yolo8s.pt	100	16	640	0.001	Adam	None
yolo8s_adamw_tuned	yolo8s.pt	100	16	640	0.0005	AdamW	Heavy
yolo8s_heavy_augmentation	yolo8s.pt	100	16	640	0.01	SGD	Heavy

Heavy augmentation included mosaic and mixup, improving robustness to occlusion and clutter.

III. EXPERIMENTS AND RESULTS

We split the experimental narrative into two phases: **Initial experiments** on the full 60-class TACO dataset, and **Refined experiments** on the reduced 18-class dataset.

A. Initial Experiments (60 classes)

Trained models on the original TACO (60 classes). Severe class imbalance impacted per-class AP.

The first phase of experimentation used the complete 60-class TACO dataset. The extreme imbalance across categories had a substantial impact on detection performance, particularly for classes with very few samples.

- Classes with minimal representation showed near-zero average precision; for example, the *Battery* category contained only two annotated instances, making reliable learning impossible.
- YOLOv8s consistently outperformed the YOLOv8n baseline in overall precision and mAP, although its validation loss occasionally fluctuated due to the unstable class distribution.
- Models trained with the Adam optimizer achieved noticeably higher recall but tended to trade off some precision compared to SGD-based runs.
- Data augmentation, including mosaic, flips, and color jitter, enhanced generalization and mitigated some imbalance effects when applied conservatively.

Figure 2 shows a schematic comparison of metrics across the 5 models.

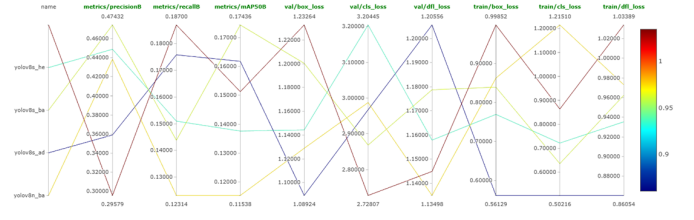


Fig. 2: Training and validation metrics (precision, recall, mAP, losses) across initial experiments.

Representative numeric results for the 60-class experiments:

TABLE II: Representative results initial experiments (60 classes)

Model	Precision	Recall	mAP50
yolo8n_baseline	0.438	0.123	0.115
yolo8s_baseline	0.449	0.151	0.138
yolo8s_adam_optimizer	0.359	0.176	0.162
yolo8s_adamw_tuned	0.474	0.144	0.174
yolo8s_heavy_augmentation	0.296	0.187	0.151

Analysis: the nano model (yolo8n) had the highest precision but low recall, it missed many litter instances. The yolo8s variants provided a better balance for deployment.

B. Refined Dataset Experiments (18 classes)

To reduce imbalance, the dataset was reduced to 18 classes using a YOLO-formatted version [5]. Models were retrained with similar hyperparameters and assessed.

- Class distribution was more even, allowing models to learn underrepresented classes.
- Overall mAP, recall, and per-class AP increased across the board.



Fig. 3: Class distribution (18-class refined dataset) [6].

Figure 4 shows a schematic comparison of metrics across the 4 models.

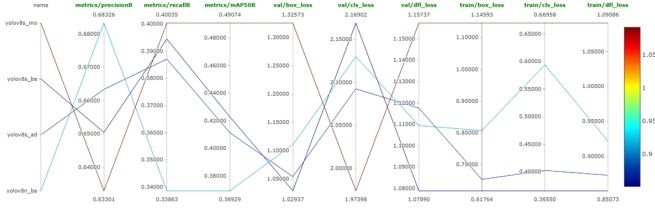


Fig. 4: Training and validation metrics (precision, recall, mAP, losses) across initial experiments.

TABLE III: Performance metrics for refined 18-class models

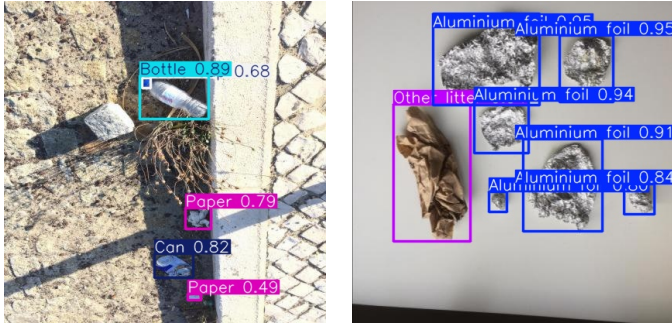
Model	mAP50	Precision	Recall
yolov8s_adam_optimizer_mod	0.411	0.663	0.387
yolov8s_baseline_mod	0.423	0.650	0.394
yolov8n_baseline_mod	0.369	0.683	0.339
yolov8s_modified8	0.491	0.633	0.400

Observations:

- The **yolov8s_modified8** model (heavy augmentation) achieved the highest mAP50 (0.49) and recall (0.40), making it the recommended deployment candidate.
- The nano model remained a precision specialist but had low recall (misses too many objects).
- Heavy augmentations increased training loss in some cases but reduced validation classification loss, indicating better generalization.

C. Qualitative Results

Sample detection outputs from the best model are shown in Figure.



(a) Detection sample 1

(b) Detection sample 2

Fig. 5: Detections by the deployed model (yolov8s_modified8).

D. Additional Diagnostic Figures

To further examine model behavior and parameters, all experiment logs, metrics, and models are available through MLflow on the DagsHub project repository [7].

IV. CONCLUSIONS AND FUTURE WORK

- Reducing class count to 18 and improving class balance produced significant gains in detection metrics.

- Heavy data augmentations (mosaic, flips, rotations) combined with YOLOv8s gave the best operational balance of recall and precision.
- Future work: collect more labeled images for rare classes, explore class-aware sampling and loss weighting, and integrate detectors into mobile or robotic cleanup systems.

ACKNOWLEDGMENTS

Experiment logs and models were tracked using MLflow/DagsHub. Thanks to dataset authors and maintainers of the YOLO ecosystem.

REFERENCES

- [1] Ultralytics, "Ultralytics yolov8 documentation," <https://docs.ultralytics.com/>, 2023, accessed: 2025-12-10.
- [2] L. Bossard, M. Guillaumin, and L. V. Gool, "Taco: Trash annotations in context," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2019, dataset: Trash Annotations in Context (TACO).
- [3] kneroma, "Taco trash dataset," Kaggle dataset, <https://www.kaggle.com/datasets/kneroma/tacotrashdataset>, 2025, accessed: 2025-12-10.
- [4] A. Syed, "Taco dataset processing notebook," <https://www.kaggle.com/code/syedabdullah1247/official-taco-dataset>, 2025, accessed: 2025-12-10.
- [5] V. Lanz, "Taco dataset (yolo format)," Kaggle dataset, <https://www.kaggle.com/datasets/vencerlanz09/taco-dataset-yolo-format>, 2023, accessed: 2025-12-10.
- [6] nujhatraisaarpa, "Taco trash-detection yolo12n notebook," Kaggle notebook, <https://www.kaggle.com/code/nujhatraisaarpa/taco-trash-detection-yolo12n>, 2025, accessed: 2025-12-10.
- [7] A. Syed, "Litter detection mlflow runs (project logs)," <https://dagshub.com/abdullahsyed2005/litter-detection.mlflow/#/experiments/7>, accessed: 2025-12-10.